

3D DATA PROCESSING - LAB 1

Stereo Matching - Report

Task 1: Path Cost Calculation

The main part of the SGM algorithm was implemented in the `compute_path_cost()` method. For each pixel along a 1D path, the algorithm computes a cost for every possible disparity d . This cost is recursive, depending on the previous pixel's cost in the same direction. I implemented the three standard SGM transitions:

- **No change:** Using the same disparity as the previous pixel.
- **Small change (P_1):** A shift of ± 1 in disparity.
- **Large change (P_2):** A disparity jump greater than 1, penalized by the sum of the best previous cost and the P_2 constant.

The final cost for a pixel at disparity d is the sum of the local matching cost (Hamming distance) and the minimum of these three transitions, normalized by the best previous cost to maintain numerical stability.

Task 2: 2D Aggregation Traversal

To achieve "Semi-Global" coverage, I implemented a **traversal logic** in the aggregation function to handle **8 distinct paths** (horizontal, vertical, and diagonal). I used ternary logic to dynamically determine the **loop bounds** (start, end) and the **step direction** (+1 or -1) based on the unit vector of each path. This ensures that the "previous" pixel is always calculated before the "current" pixel, maintaining **causality** for all directions regardless of whether the scan moves forward or backward across the image grid.

Task 3: Disparity Sample Collection

Standard SGM often results in invalid regions or missing data in textureless areas or zones of occlusion where the matching confidence falls below the required threshold. To bridge this with monocular depth, I implemented a **data collection step**. For every pixel where SGM achieved a high-confidence match (determined by the `inv_confidence_` threshold), I stored a pair consisting of the **trusted SGM disparity** and the **raw monocular intensity** at the same coordinate. These pairs serve as the training data to calibrate the unscaled monocular map.

Task 4: Monocular Scale Estimation and Refinement

Using the samples from Task 3, I implemented a linear regression model:

$$Disparity_{SGM} = h \cdot Intensity_{Mono} + k$$

I used the Eigen library to solve this overdetermined system using LDLT decomposition (Least Squares).

- **Scale Factor (h):** Stretches the monocular values to the physical pixel shift range.
- **Offset (k):** Aligns the baseline of the two sensors.

In the refinement pass, I identified low-confidence pixels and replaced their values with this calibrated monocular estimate, resulting in a smooth and physically accurate disparity map.

Task 5: Adaptive Penalties

For the adaptive penalty option, I chose to implement a **dynamic threshold switch**. This decision was driven by the observation that the primary challenge in the constant penalty maps was the presence of blurring artifacts rather than a general lack of edge accuracy. I calculated the **global mean gradient** of the reference image to establish a baseline for contrast. This global average serves as the foundation for the **dynamic threshold**; by anchoring the switch to this value, the threshold automatically scales based on the overall lighting and texture density of each specific dataset.

- **Flat Regions:** If the local gradient is below $2.0 \times$ the mean, I enforce high penalties ($P_1=10$, $P_2=60$) to maintain smoothness.
- **Edge Regions:** If a high gradient is detected, I assume a physical boundary and drastically reduce the penalties ($P_1=1$, $P_2=20$).

This enabled the disparity map to better conform to object boundaries, resulting in sharper depth discontinuities and a reduction in blurring artifacts.

Experimental Results and Analysis

To evaluate the performance of the algorithm at each stage of development, the Mean Squared Error (MSE) was calculated against the ground truth for all four datasets. The table below illustrates the significant reduction in error achieved through monocular refinement and the further optimization provided by adaptive penalties.

Data Item	Aloe	Cones	Plastic	Rocks1
MSE before refinement	122.464	475.166	820.049	557.735
MSE after refinement	13.7291	17.4342	348.223	34.6984
MSE adaptive penalties	13.3108	14.2728	22.6812	33.747

To visually evaluate the progression of the algorithm, the disparity maps are displayed below in the following order (from left to right): Before Refinement, After Monocular Refinement, and finally, the output using Adaptive Penalties.

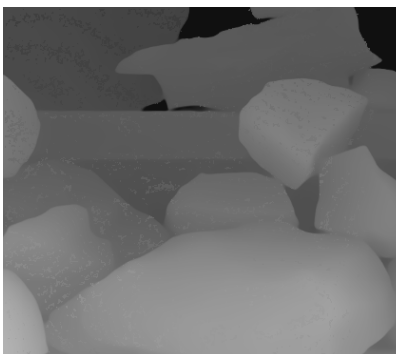
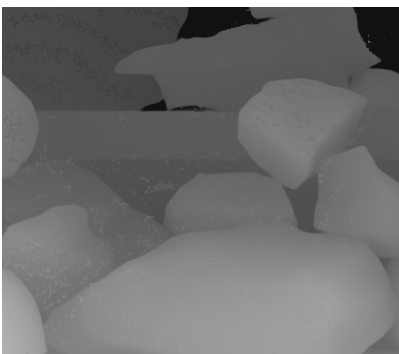
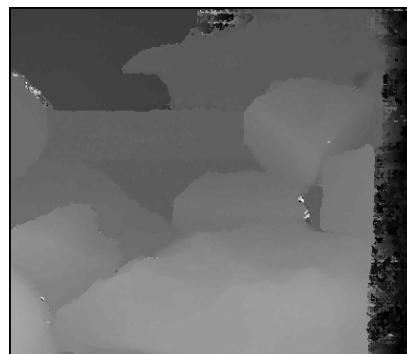
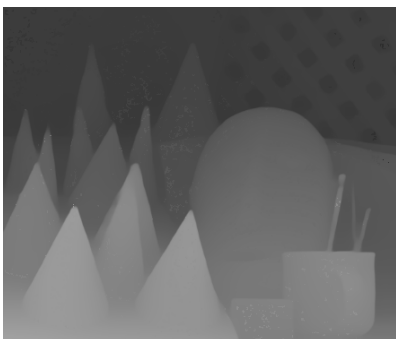
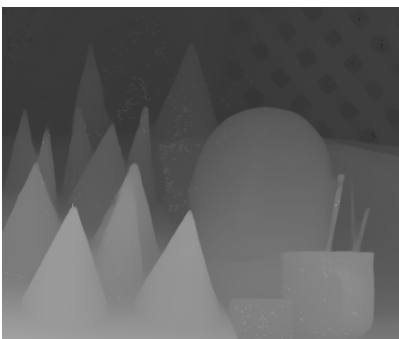
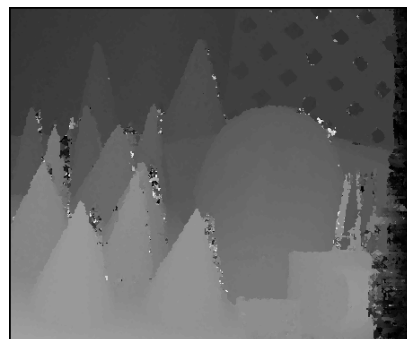
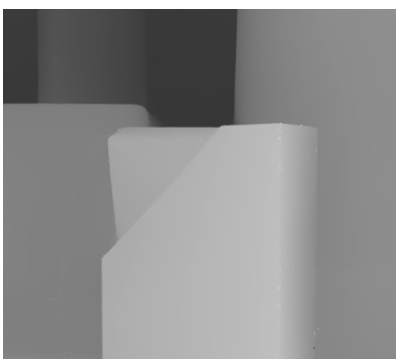
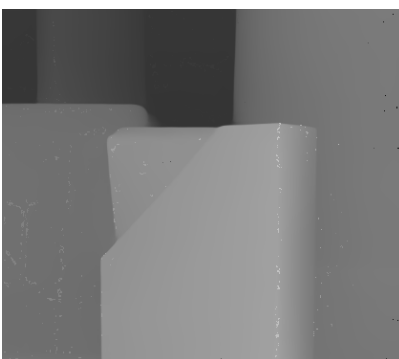
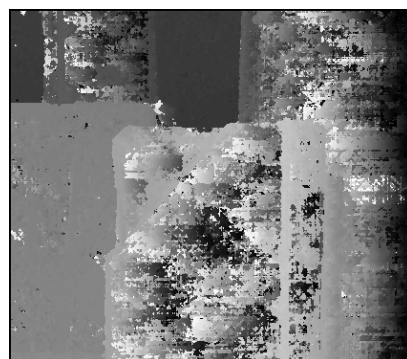
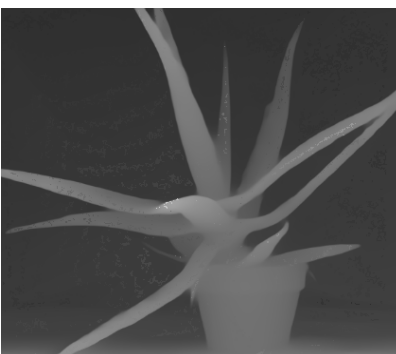
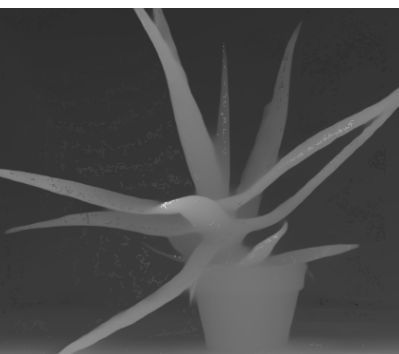
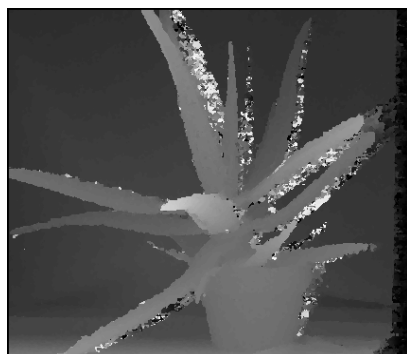
The maps showing the outputs **before refinement** are quite noisy and incomplete because the algorithm only keeps pixels where it has high confidence in the match. You can see that while the main shapes (like the leaves of the Aloe or the blocks in Plastic) are starting to form, there are many 'holes' and mismatched pixels. This lack of density is what led to the high initial MSE values seen in the quantitative analysis.

The **qualitative improvements** provided by the final implementation stages are evident in the resulting disparity maps. Tasks 4 and 5 effectively addressed the previously discussed sparsity.

Before Refinement

After Refinement

Adaptive Penalties



- **Plastic:** This dataset showed the most significant qualitative and quantitative improvement. By utilizing the adaptive switch, the algorithm resolved the previous "bleeding" artifacts, leading to a massive reduction in MSE.
- **Rocks1:** Due to the complex, high-frequency textures inherent to rocky surfaces, achieving a substantial improvement with the adaptive penalties was more difficult. The dense texture often creates a high-gradient signal that can occasionally interfere with the algorithm's ability to distinguish between surface texture and true depth jumps.
- **Aloe and Cones:** These images reached a very low error rate with constant penalties. Given the organic patterns and relatively soft gradients, the baseline was already near-optimal after the refinement, resulting in a marginal numerical gain with the adaptive approach.

Problems Encountered and Discussion

- **Computational Efficiency:** Initially, I presumed that the implementation would be restricted to the commented regions for each task, but calculating the mean gradient of the reference image within the innermost pixel loop caused a significant bottleneck in runtime. To optimize performance, I moved this calculation to the `set()` method. This choice felt appropriate since the purpose of pre-computing this global image statistic was to provide a baseline for the adaptive heuristic. This allows the local gradient at each pixel to be evaluated relative to the typical edge strength of the entire image, ensuring consistent sensitivity across different scenes.
- **Heuristic Selection:** A significant portion of the development was spent experimenting with the adaptive penalty function. I tested several mathematical models, including linear scaling, quadratic, cubic, and exponential decay functions. While these continuous models are theoretically more flexible, empirical testing showed that they often introduced subtle noise in textureless regions. Ultimately, the dynamic threshold switch proved to be the most robust approach, as it provided a clear separation between smoothing and edge-snapping behaviors.
- **Generalization and Dataset Variability:** While the chosen dynamic threshold and penalty values yielded positive results across the four provided datasets, it is important to note that these parameters may not generalize perfectly to all possible stereo pairs. However, since the test images (ranging from the high-contrast geometric shapes in *Plastic* to the organic, soft-textured patterns in *Aloe*) represented a wide variety of visual scenarios, the heuristic demonstrated a high degree of adaptability. While I cannot guarantee equal performance on entirely different datasets, the current results suggest that the logic is robust enough to handle a diverse range of photometric and geometric conditions.
- **Implementation Workflow:** Beyond these specific hurdles, the implementation was a direct translation of the provided theoretical slides into C++ code. The primary task was not troubleshooting the theory itself but rather determining the most effective way to translate those concepts into a performant software architecture.